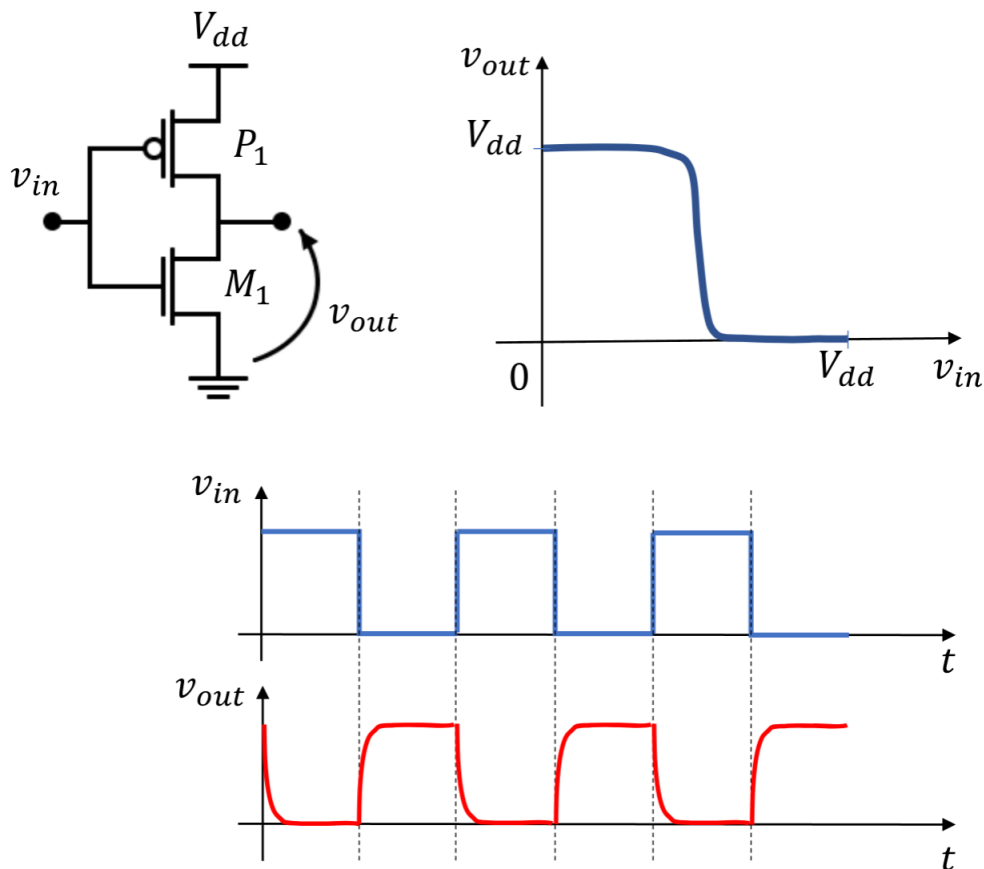


1 Lesson 25-09-26

In the last lesson we introduced the resistive load inverter. We said that the main problem of such a design is the asymmetric response, and static power consumption (as in the output off state we have a closed path from power source to ground, draining power on a resistor). Today we try to solve those problems.

CMOS inverter: a better inverter can be designed by using **Complementary MOS** (CMOS) designs.



We see that in such a design:

- The response is **symmetric** (or mostly symmetric, we can act on the gate sizes to account for mobility effects);
- There is no **static** power consumption (in the ideal case, some leakage does happen);
- **Dynamic** power is present and measurable based on equivalent capacitance of the wire and the MOS transistors:

$$P \approx C_{e1} f V^2$$

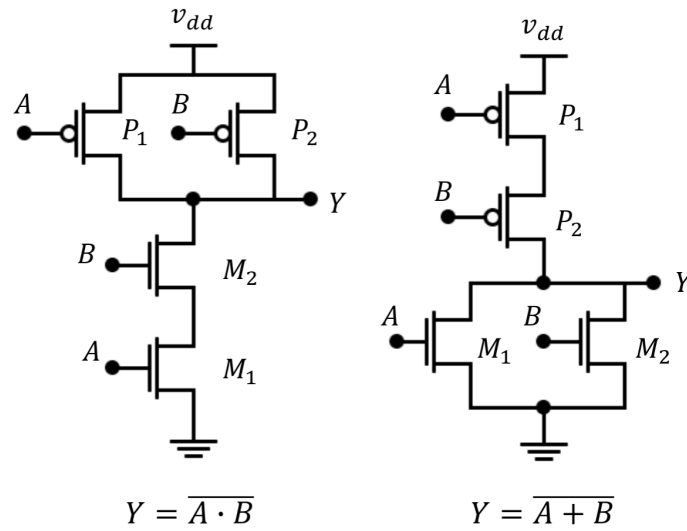
This design is very common in digital circuits nowadays, as it performs much better than the resistive load inverter.

CMOS logic: other logic functions are needed to implement a logic family. For CMOS technology, we can use combinations of transistors in series or in parallel to implement any kind of logic.

In particular:

- In the pull-down branch:
 - Series NMOS implements a NAND
 - Parallel NMOS implements a NOR
- In the pull-up branch:
 - Series PMOS implements a NOR
 - Parallel PMOS implements a NAND

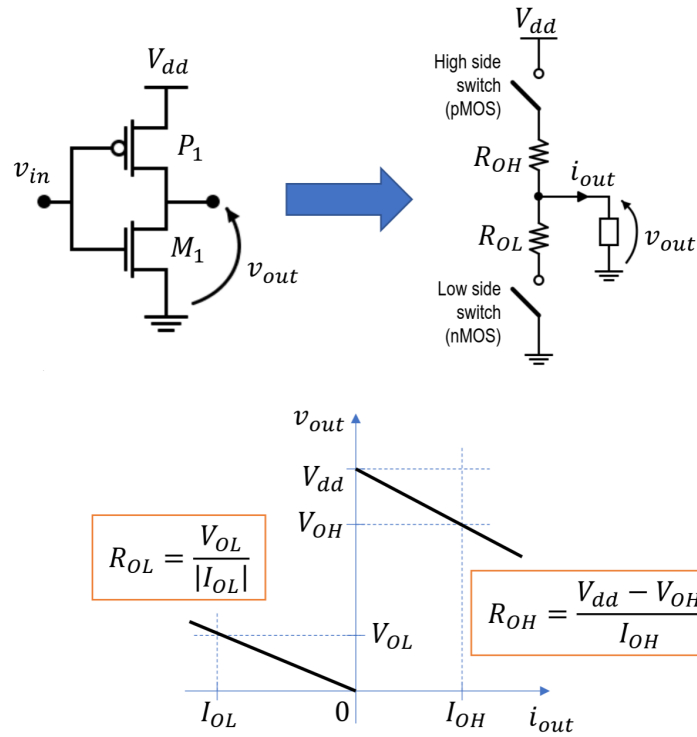
We show a figure of the synthesis of a NAND and NOR circuit in CMOS technology:



Logic gate parameters: we can define input and output parameters for logic gates, regarding both *current* and *voltage*.

With regards to **current**, we can model a CMOS inverter converting the MOS transistors to voltage controlled switches (ideal) with series resistances modeling leakage. This way we see we can define the parameters:

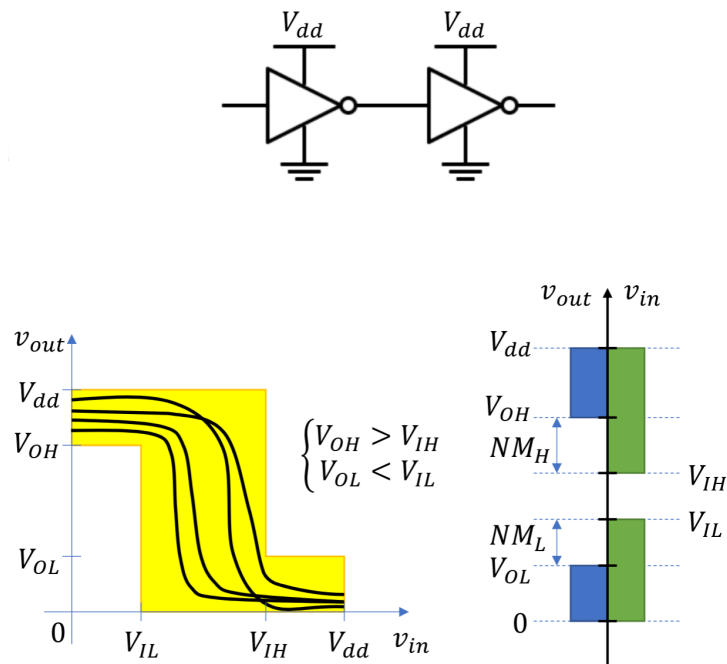
- V_{OH} , which is the lowest output voltage to get a logic high output;
- V_{OL} , which is the highest output voltage to get a logic low output;
- I_{OH} , which is the maximum output current such that the output voltage at logic high is greater than V_{OH} ;
- I_{OL} , which is the maximum output current such that the output voltage at logic low is less than V_{OL} .



From these parameters (given by constructors) we can compute the value of the modeling resistances:

$$R_{OL} = \frac{V_{OL}}{|I_{OL}|}, \quad R_{OH} = \frac{V_{dd} - V_{OH}}{I_{OH}}$$

Now, with regards to input and output **voltages** in stacked gate setups, we can consider the following:



Here we have a pair of connected inverters. We are interested on whether output voltages of the first inverter will be correctly interpreted at the input of the second inverter, so we define the following parameters:

- V_{IH} , which is the lowest input voltage recognized as logic high;
- V_{IL} , which is the highest input voltage recognized as logic low;

From this we can derive stability *conditions* for the whole logic family:

$$V_{OH} > V_{IH}, \quad V_{OL} < V_{IL}$$

where we define the following margins:

$$V_{OH} - V_{IH} = NM_H \quad (\text{noise margin high})$$

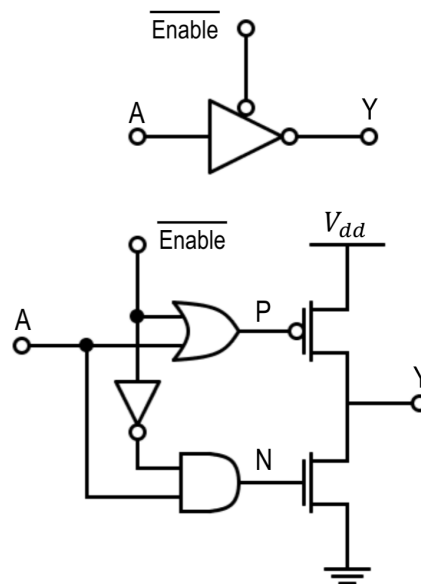
$$V_{IL} - V_{OL} = NM_L \quad (\text{noise margin low})$$

Tri-state outputs are needed to avoid logic gates shorting when connected to the same outputs.

In these designs we have an additional Enable input, which can be *active high* or *active low*:

- When Enable is active, the output is driven normally;
- When Enable is NOT active, both the pull-up (high side) and pull-down (low side) transistors are switched OFF, and the output line is left floating. At this point the voltage level (and consequently the logic level) is set by some other circuit, or remains undetermined.

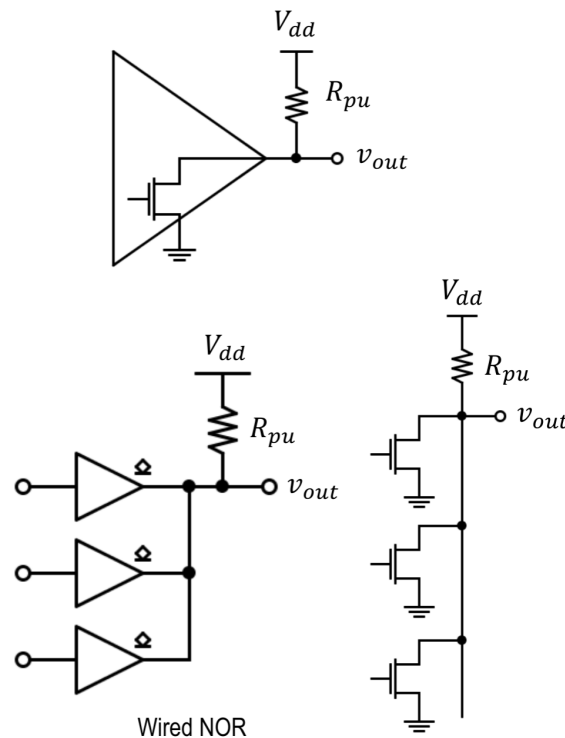
Below, we show the symbol of a tri-state inverter and a possible CMOS implementation.



As we said, tri-state outputs are useful when multiple logic gates drive the same line, for instance in a shared bus. All drivers must be in high impedance except one: it is vital to guarantee the above condition to avoid short circuits.

Open collector is another technology which allows to drive the same data line, even without knowing the voltage supply value.

In this setup, there is no pull-up transistor at the output: we can only set the output to logic 0. To set a logic 1, we switch off all pull-down transistors, and use an external pull-up resistor.



This is useful to drive the same line with multiple logic gates:

- No possible short circuit: when all logic gates set a logic 1 the output is at logic 1;
- When at least one logic gate sets a logic 0, the output goes at logic 0. This makes open collector a NOR logic function.

This is slightly easier to implement than the earlier tri-state solution, as we don't have to deal with high impedance or short circuit issues.

Of course, we can imagine an *open drain* setup which works the same, only leaving the pull-down transistor outside of the design and pulling *up* through an external resistor.

1.1 Combinatorial circuits

Combinatorial circuits are the simplest we can see with digital logic circuits. In a combinatorial logic circuit, the logic output depends on the current value of the inputs only (no state).

A combinatorial circuit can be described through a **truth table**, which is a list all possible input combinations, and corresponding output values.

Many ways exist to synthesize a combinatorial circuit:

- The main idea is generally to generate the circuit as a **SoP** (*Sum of Products*), minimizing the number of products, and minimizing the number of literals in each product (which corresponds to minimizing components).
- A possible tool is given by *Karnaugh maps*;
- Another possibility is the *Quine-McCluskey algorithm* for more inputs;

- Often, we just stick to logic synthesis *software tools*, which implement these one other advanced algorithms.

Multiplexers are circuits which allow to select an input and output just one of them, between different inputs. A one bit multiplexer with 2 inputs has this boolean law:

$$y = a \cdot s + b \cdot \bar{s}$$

where a and b are the inputs, and s is the selection signal.

For multiplexers on multiple bits, we repeat the same structure multiple times; For multiplexers with more than two inputs, we need more than one selection signal (usually built hierarchically with log number of inputs smaller multiplexers). Multiplexers with more than two inputs on multiple bits are built as a combination of the above two cases.

Decoders decode the binary representation of a number, on n bits, to the same number in one-hot representation, on 2^n bits.

Encoders do the reverse job of converting a one-hot representation on 2^n bits in a single n bit encoded value.

More complex components like *priority encoders*, ... can be built.

Digital adders / subtractors are built in combinatorial logic too.

We start from *half adders*: these operate on 1-bit operands, generating a carrying output bit. For *multibit* adders we combine multiple half adders.

Different area and performance can be achieved based on carry computation. For instance, we remember:

- Ripple Carry
- Carry Look Ahead
- Carry Select

Subtractors are just implemented using an adder (using 2-complement notation).

1.2 Sequential circuits

Sequential circuits need a *memory element* to exist somewhere in a circuit. The most popular memory element is given by a pair of active elements in a loop.

- A pair of inverters in a loop provides a memory element, that is an element that is *bistable*: either of the inverters can start out with an high outputs, and the state is converted. The problem of this design is that we can't control it:
- A pair of NANDS in a loop can be used in the same fashion. The input bits that are left out can be used as SET and RESET parameters for the bistable circuit. We call this a **SR latch**.